Feedback de aulas

Tech Challenge - Módulo 4 | Documentação do projeto 09 de novembro de 2026

1. Objetivo

O projeto permite receber avaliações de aulas, identificar situações que precisam de atenção e acompanhar os resultados em um relatório semanal. Foi desenvolvido em Java 21 e utiliza serviços do Microsoft Azure.

A entrada é uma API: um endereço que recebe dados enviados por ferramentas como o Postman. A solução não inclui uma tela própria para o aluno preencher a avaliação.

2. Funcionamento em três etapas

1. Receber a avaliação: a função ReceberAvaliacao verifica a descrição e a nota, classifica a urgência e salva os dados no Azure Table Storage.

2. Enviar o alerta: quando a avaliação é crítica, uma mensagem vai para uma fila. A função EnviarAlerta lê essa mensagem e solicita o envio de e-mail ao administrador.

3. Gerar o relatório: a função GerarRelatorioSemanal consulta a semana anterior, calcula os resultados e salva um arquivo JSON no Blob Storage.

A fila funciona como uma lista de mensagens a processar. Ela separa o registro da avaliação do envio do e-mail, permitindo responder à chamada antes da conclusão do envio.

3. Tecnologias utilizadas

Tecnologia	Uso no projeto
Java 21 e Maven	Linguagem e ferramenta para compilar, testar e empacotar.
Azure Functions	Executa funções por requisição, fila ou horário.
Table, Queue e Blob Storage	Guardam avaliações, mensagens e relatórios.
Communication Services Email	Serviço utilizado para enviar os alertas.
Application Insights	Permite acompanhar execuções e investigar falhas.

Regras e uso da API

4. Dados obrigatórios

A descrição deve ser um texto não vazio, com até 2.000 caracteres. A nota deve ser um número inteiro de 0 a 10. Valores decimais e notas enviadas como texto não são aceitos.

Nota	Classificação	Ação
0 a 3	CRITICA	Salvar e encaminhar alerta para a fila.
4 a 6	ATENCAO	Salvar para acompanhamento.
7 a 10	NORMAL	Salvar para acompanhamento.

Essas faixas são as regras adotadas pelo projeto. Cada registro recebe um identificador e a data de envio.

5. Exemplo de chamada

Método: POST | Caminho: /api/avaliacao

Cabeçalhos: Content-Type: application/json e x-functions-key com a chave da função. No Azure, a origem também precisa estar no IP autorizado.

```
{"descricao":"Aula bem explicada","nota":9}
```

Exemplo de resposta após o registro:

```
{"id":"identificador-gerado","urgencia":"NORMAL"}
```

Resposta HTTP	Significado
201	Avaliação registrada com sucesso.
400	Dados inválidos, como nota fora do limite.
401	Chave ausente ou inválida na origem permitida.
403	Pode indicar bloqueio por restrição de IP.
500	Falha interna ao registrar a avaliação.

A resposta 201 confirma o registro. Ela não confirma a entrega de um e-mail. O alerta é processado separadamente.

Relatório e segurança

6. Relatório semanal

O agendamento configurado é segunda-feira às 12:00 UTC, equivalente a 09:00 de São Paulo. O relatório considera a semana anterior, de segunda a domingo, no fuso America/Sao_Paulo.

O arquivo contém o período, o total de avaliações, a média das notas, as contagens por dia e por urgência, além das avaliações com descrição, nota, urgência e data de envio.

A média é a soma das notas dividida pelo número de avaliações. Dias sem registros continuam aparecendo com zero. Se a semana estiver vazia, a média será null, indicando que não existe uma média a calcular.

Exemplo registrado na validação

Na amostra fictícia de 19 a 25 de outubro de 2026, foram 12 avaliações, com soma de notas 72. Portanto, a média foi 72 ÷ 12 = 6,0. A distribuição foi: 3 críticas, 3 de atenção e 6 normais.

Esses valores pertencem à amostra histórica documentada. Não representam uma consulta atual ao Azure. As avaliações fictícias do histórico foram inseridas administrativamente e não dispararam e-mails.

7. Proteção de acesso

A API exige chave de função e permite apenas o IP autorizado. O endereço continua público na internet, mas o acesso é restrito. Essa configuração não equivale a uma rede privada ou VPN.

Os relatórios ficam em um container sem acesso público. A aplicação usa identidade gerenciada, uma identidade fornecida pelo Azure, para acessar o armazenamento com as permissões atribuídas.

As configurações de e-mail ficam no ambiente da aplicação. Chaves e valores de conexão não devem ser incluídos no código nem exibidos em gravações. Os exemplos do projeto usam dados fictícios.

8. Monitoramento e limites

O Application Insights registra execuções e falhas. Os logs da aplicação utilizam identificadores e contagens, evitando incluir descrições e destinatários.

Salvar a avaliação e publicar na fila são operações separadas. Uma falha pode deixar um registro salvo sem alerta. O reprocessamento de mensagens também pode duplicar e-mails. Após cinco falhas do consumidor, a mensagem vai para a fila alertas-criticos-poison para análise.

O relatório carrega os dados em memória e foi pensado para o volume acadêmico. Melhorias futuras incluem evitar duplicações e reforçar a consistência entre o registro e o alerta.

Execução e validação

9. Como verificar o projeto

Pré-requisitos: Java 21 e Maven. Na pasta do projeto, execute:

```
mvn clean verify
```

Esse comando compila o código, executa os testes e gera o pacote das funções. O resultado fica em target/azure-functions/tech04-local.

Para executar localmente, também são necessários Azure Functions Core Tools v4 e Azurite, que simula o armazenamento. Copie local.settings.example.json para local.settings.json, inicie o Azurite e crie a tabela avaliacoes. Depois execute:

```
mvn azure-functions:run
```

No arquivo de exemplo, o envio de alertas e o relatório agendado começam desabilitados. O e-mail depende das configurações EMAIL_CONNECTION_STRING, EMAIL_SENDER e ADMIN_EMAIL e dos serviços correspondentes. O ambiente local não aplica as chaves HTTP como o ambiente em nuvem.

10. Publicação no Azure

O projeto inclui infraestrutura em Bicep e o script scripts/deploy.ps1 para testar, preparar os recursos e publicar a aplicação. A publicação exige Azure CLI autenticada, permissões na assinatura, uma região compatível e o IP autorizado. A configuração de e-mail é feita por scripts/configure-email.ps1.

Esses scripts alteram recursos reais e podem consumir créditos da assinatura. Não é necessário executá-los novamente apenas para consultar a documentação.

11. Resultados documentados

O registro de validação do projeto informa seis testes JUnit sem falhas nem erros. Eles cobrem validação, classificação, encaminhamento de alertas, falha no armazenamento, limites da semana e cálculos do relatório.

Também estão documentados o reconhecimento das três funções no Azure, respostas HTTP 401, 400 e 201, e a geração de relatórios em armazenamento privado. O ambiente registrado utiliza Azure for Students, grupo rg-tech04-academico e região France Central.

As evidências descritas são de validações anteriores. Esta documentação foi preparada a partir dos arquivos do projeto; não foi feita uma nova consulta ao ambiente em nuvem para sua elaboração.

12. Organização e fontes do projeto

src/main/java: código da aplicação. src/test/java: testes. infra: infraestrutura. scripts: automação e demonstração. docs: evidências e consultas. README.md: visão geral e instruções.

Base desta documentação: README.md, docs/validacao.md, pom.xml, local.settings.example.json e classes Java do projeto.